

[L0-L1 Core Essence & Design Logic]

L0 Essence: : Maintain a replicated state machine with strong consistency across distributed nodes via the Raft consensus protocol, backed by an MVCC multi-version store and BoltDB, constructing a highly available coordination and service discovery engine for cloud-native infrastructures.

L1 Logic:

- :
1. **Raft Replicated State Machine** (M1) guarantees log replication and status commits across nodes, utilizing ReadIndex/LeaseRead to eliminate dirty reads during partitions.
2. **WAL & Embedded BoltDB** (M2) serve as the storage backbone, where WAL sequential flushes guarantee durability and BoltDB's B+ tree manages key-value states.
3. **MVCC & KeyIndex** (M3) enable transaction isolation and historic version tracking, resolving write-write conflicts via revision tracking and supporting compaction.
4. **Lease & Watch** (M4 & M5) provide dynamic leasing and asynchronous event streaming, serving as the nervous system for distributed locks and configuration dispatch.

[L2 Core Coordination Data Flow Topology]

:
• [User gRPC Request] → [Raft Log Consensus] → [Sequential WAL Write] → [Raft Commit & Apply] → [Memory KeyIndex Update] → [Batch Commit to BoltDB] → [WatchableStore Queue Update] → [WatchStream Multiplexed Dispatch] → [LeaseManager TTL Cleanup]

[M1.1] Raft State Machine & Term

- **Role Transitions:** Nodes transit between **Follower**, **Candidate**, and **Leader** roles. The **Term** acts as a logical clock, incremented with each election.
- **Term Constraints:** Any node receiving messages with a smaller **Term** discards them; receiving a message with a larger **Term** forces the node to step down to **Follower** and synchronize its local **Term**.

[M1.2] Leader Election & Randomized Timeout

- **Randomization:** **Election Timeout** is randomized (e.g., 1000ms - 3000ms) to prevent split votes where multiple **Candidates** split the quorum, causing deadlocks.
- **Election Safety:** A **Candidate** must receive votes from a majority (**Quorum**) to win. A voter only votes for **Candidates** whose log is at least as up-to-date as its own (comparing **Terms** first, then log **Index**), ensuring the elected **Leader** contains all committed logs.

[M1.3] Log Replication & Commit

- **Three-Phase Pipeline:** Leader receives a write → appends log entry locally → broadcasts **AppendEntries** to **Followers** → upon receiving **Quorum** success, marks the log as committed and applies it to the state machine.
- **State Tracking:** Leader maintains **MatchIndex** (highest index known to be replicated) and **NextIndex** (index of next log entry to send) for each **Follower**, dynamically updating indices via fallback probes.

[M1.4] Network Partition & Pre-Vote Phase

- **Partition Pain:** A partitioned **Follower** cannot hear heartbeats, causing it to increment its **Term** and trigger elections. Upon reconnection, its high **Term** forces the active **Leader** to step down, disrupting the cluster.
- **Pre-Vote Validation:** **Follower** runs a mock "Pre-Vote" phase without incrementing its **Term**. It only escalates to **Candidate** if the **Quorum** agrees it has an up-to-date log, preventing split-brain disruption.

[M1.5] Membership Changes & Joint Consensus

- **Transition Risks:** Direct configuration switches ($C_{old} \rightarrow C_{new}$) can cause split-brain if different nodes activate new memberships at different times, allowing overlapping majorities.
- **Joint Consensus:** etcd uses a transition state $C_{old,new}$. Proposals must win majorities in both C_{old} and C_{new} separately before committing, after which the transition to C_{new} completes safely.

[M2.1] WAL Append-Only Structure

- **I/O Alignment:** WAL files are preallocated to 64MB using `fallocate` to bypass filesystem disk block allocation delays and random disk writes.
- **Sector Alignment:** Log writes are aligned to 512 bytes (physical sector size) and written sequentially, maximizing write throughput.

[M2.2] BoltDB/bbolt B+ Tree Storage

- **B+ Tree Layout:** bbolt maps B+ tree pages directly to a single file. Leaf nodes store actual key revisions and value payloads, while interior nodes store index pointers.
- **Zero-Copy Memory Map:** Leverages `mmap` to map the db file directly into memory, allowing read transactions to run zero-copy without copying data from kernel to user buffers.

[M2.3] WAL & BoltDB Commit Sequence

- **Sequential Barrier:** Write requests are **always appended to the WAL and synced via fsync first**, then committed to BoltDB.
- **Crash Recovery:** If the system crashes during BoltDB write phases, etcd replays the WAL log on startup to recover uncommitted state machine modifications.

[M2.4] Snapshotting & WAL Truncation

- **Memory Deflation:** To prevent WAL from growing indefinitely, etcd takes periodic snapshots of BoltDB states.
- **Log Recycler:** Once a snapshot is flushed to disk, WAL entries older than the snapshot's **Revision** are safely truncated to reclaim disk space.

[M3.1] Revision & Generation Lifecycle

- **Revision Counter:** etcd uses a 64-bit global incrementing revision index. Each mutation (PUT/DELETE) increments this counter by 1.
- **Sub-Revision Struct:** Revisions consist of `main` (write transaction sequence) and `sub` (internal transaction offset index, starting at 0), serving as a unique pointer to any KV modification.

[M3.2] KeyIndex & In-Memory B-Tree

- **Index Mapping:** Since BoltDB keys are revisions, etcd maintains an in-memory B-Tree mapping user keys to their revisions (**KeyIndex**).
- **Generation Struct:** Each **keyIndex** contains a `generations` array. A **Generation** tracks the life of a key from creation (PUT) to deletion (DELETE), recording all revisions associated with its changes.

[M3.3] MVCC Multi-Version Read Isolation

- **Lock-Free Read Snapshot:** Reading without specifying a revision queries the latest revision. Querying with `--rev=100` searches **KeyIndex** for revision 100, then reads the data from BoltDB.
- **Read-Write Concurrency:** Since older versions remain in BoltDB, read transactions require no lock and can read historic snapshots, allowing non-blocking reads and writes.

[M3.4] Compaction & DB Defragmentation

- **Revision Compaction:** MVCC database growth requires regular **Compaction** to purge historical revisions before a specified threshold.
- **Defragmentation:** **Compaction** marks BoltDB pages as free but does not return space to the OS. `etcdctl defrag` reorganizes database files to physically reclaim disk space.

[🔗] etcd Architectural Trade-offs & Fallacies Matrix

Follower reads are fast and scale linearly.: Reading from a **Follower** can return stale data due to replication lag; enabling **Linearizable Reads** requires roundtrips to the **Leader** for **Quorum** heartbeats, adding RTT. → Enable **ReadIndex** by default to prevent disk writes for reads; use **LeaseRead** in environments with stable clocks (drift under safety margin) for 0 RTT reads; use **Serializable** reads for non-critical high-frequency configurations.

Watcher will receive all historical updates if restarted.: If the requested revision is compacted, the **Watcher** fails to start and returns an `errCompacted` error. → Monitor compaction progress; implement a client retry logic where if `errCompacted` is caught, it refetches the baseline snapshot, resets state, and resumes watching from the latest revision.

Creating short-lived independent leases for keys is simple.: Every independent lease incurs **Min-Heap** sorting costs. Thousands of active leases cause high CPU usage and double-ended **KeepAlive** streaming network storms. → Implement "Lease Sharing." Bind thousands of keys with similar lifespans to a single shared lease ID, consolidating heartbeats into one multiplexed socket renewal stream.

[M4.1] Lease Manager & Lease Map

- **Min-Heap Queue:** The `LeaseManager` schedules lease expirations using a **Min-Heap priority queue** ordered by absolute expiration time ($O(1)$ time complexity to find expired leases).
- **Fast ID Lookup:** Maintains a hash map of `Lease ID` to lease structures for immediate lookup during keepalives and revocations.

[M4.2] KeepAlive & gRPC Bidirectional Stream

- **Multiplexed Connection:** Instead of sending individual HTTP requests for every lease renewal, etcd leverages gRPC bidirectional streaming.
- **Bandwidth Savings:** Heartbeats are sent over a single TCP connection, drastically reducing network roundtrips and CPU overhead.

[M4.3] Lease Revocation & Key Expiration

- **Lessor Thread:** Once the lessor thread detects an expired lease ($TTL \leq 0$), it initiates a deletion transaction.
- **Cascade Revocation:** Extracts all keys bound to the Lease ID and issues delete operations to the Raft state machine, removing keys from BoltDB and notifying Watchers.

[M4.4] Lease Sharing & Scalability

- **Ephemeral Scalability:** In Kubernetes, thousands of Pods might require leases. Creating unique leases for each client increases network heartbeats.
- **Heartbeat Consolidation:** etcd encourages binding multiple keys to a single Lease ID (e.g., a shared 60s lease), consolidating heartbeats and saving memory.

[M4.5] Distributed Locks via Lease

- **Atomic Claim:** Distributed locks use atomic PUT transactions tied to a lease. A lock is held by creating an ephemeral key bound to a Lease ID.
- **KeepAlive Watchdog:** A background thread (watchdog) renews the lease. If the lock holder crashes, the lease expires and the lock key is deleted, preventing deadlocks.

[M5.1] Watcher & WatchStream

- **Event Pipeline:** Clients stream events by creating a `Watcher` on key ranges. These watchers are multiplexed onto a single gRPC `WatchStream` connection.
- **Event Payload:** Applications applying writes convert mutations into Watch Events containing details of the action, key, new value, and previous value.

[M5.2] WatchableStore synced/unsynced Queues

- **Buffer Isolation:** `WatchableStore` manages two watcher queues in memory: `synced` (up-to-date watchers) and `unsynced` (watchers catching up on historical revisions).
- **Queue Promotion:** Once a watcher catches up to the current revision, it is promoted from `unsynced` to `synced` for real-time notification.

[M5.3] Event Buffer & Flow Control

- **Memory Exhaustion Guard:** Slow clients can cause event backlogs in memory. etcd allocates a fixed ring buffer for each watcher.
- **Backpressure Drop:** If a watcher's buffer overflows, etcd terminates the stream and returns `errCompacted`, forcing the client to reconnect and catch up.

[M5.4] Watch Event Consolidation

- **Network Optimization:** Under heavy write pressure, a key may be mutated multiple times in milliseconds. Sending every event degrades throughput.
- **Debounce Updates:** etcd consolidates multiple modifications to the same key within the same transaction batch (main revision), dispatching only the final state to reduce network overhead.

[M5.5] Watch Resume via Revision

- **Disconnect Recovery:** If a watcher disconnects, it can reconnect using the last received revision (e.g., `Watch(key, WithRev(last_received_rev + 1))`).
- **History Fetch:** etcd registers the watcher in `unsynced`, fetches missing events from BoltDB historical revisions, sends them, and moves it to `synced`.

[M6.1] gRPC Protocol v3 Architecture

- **Protocol Overhaul:** etcd v3 uses gRPC over HTTP/2 with `Protocol Buffers` instead of v2 HTTP/1.x JSON.
- **System Improvements:** HTTP/2 multiplexing removes TCP connection handshakes, and Protobuf binary serialization reduces CPU and memory footprint, boosting throughput.

[M6.2] Linearizable Reads & ReadIndex

- **Dirty Read Guard:** Reading directly from Followers can return stale data. Linearizable reads guarantee the latest global state.
- **ReadIndex Broadcast:** Followers request the latest `ReadIndex` from the Leader. The Leader verifies its status by broadcasting heartbeats (Quorum check). Once the Follower's local `ApplyIndex` reaches the `ReadIndex`, the Follower returns the read payload.

[M6.3] LeaseRead Optimization

- **RTT Reduction:** `ReadIndex` requires network heartbeats to verify the Leader, introducing network roundtrip latency (RTT).
- **Lease Validation:** If local clocks are stable, `LeaseRead` allows the Leader to serve reads locally without broadcasting heartbeats, provided its lease duration is active (as no new Leader could have emerged).

[M6.4] Client Load Balancing & Failover

- **Decentralized Routing:** Clients discover cluster endpoints and manage client-side retries.
- **Automatic Handover:** The client fetches the cluster node list and caches it. If the connected node fails, the client instantly reconnects to another healthy node.

[M6.5] Etd Gateway & Proxy Mode

- **Connection Fan-in:** Thousands of clients connecting directly to etcd can exhaust TCP file descriptors (FDs).
- **Edge Concentrator:** Deploying `etcd_gateway` or `etcd_grpc-proxy` aggregates client traffic, multiplexing connections over a few long-lived sockets to the core etcd cluster.

🔧 Zone T: etcd Performance & Diagnostic Labs

T1 etcd Tuning Parameters Reference

```
;- - (- - -> -;-)
;- -
--max-request-bytes (1.5 MB -> 1.5 MB ~ 10 MB)
Limits maximum RPC packet size to prevent memory spikes and Raft network congestion.
--quota-backend-bytes (2.0 GB -> 2.0 GB ~ 8.0 GB)
Database storage quota. Reaching this triggers a write block; requires compaction.
--heartbeat-interval (100 ms -> 100 ms ~ 250 ms)
Frequency of Raft heartbeats. Adjust upward for cross-region networks to avoid flap.
--election-timeout (1000 ms -> 1000 ms ~ 3000 ms)
Election timeout. Must be 5 ~ 10 times the heartbeat interval to prevent split votes.
```

T2 etcdctl Cheat Sheet

```
• 1. Basic Read/Write & History Queries
bash
etcdctl put /prod/config/db_host "10.0.0.5" --lease=12345678abcdef
etcdctl get /prod/config/db_host
etcdctl get /prod/config/db_host --rev=45
etcdctl get /prod/config/ --prefix

• 2. Watch Events & Resume
bash
etcdctl watch /prod/config/ --prefix --hex
etcdctl watch /prod/config/ --prefix --rev=60

• 3. Lease Management
bash
etcdctl lease grant 60
etcdctl lease keepalive 12345678abcdef
etcdctl lease revoke 12345678abcdef

• 4. Maintenance & Defrag
bash
etcdctl endpoint status --write-out=table
etcdctl compact 200
etcdctl defrag
etcdctl snapshot save /var/backups/etcd-snapshot.db
```